Chapter 14 (Interlude: Memory API) — Homework (Code)

Zhiyuan Pan

Code: `code/chapter14/`. `make` builds every program with `-g` (debug symbols), and `code/chapter14/run.sh` runs each one normally, under `gdb`, and under `valgrind`.

Test machine: Ubuntu 24.04 VM, Linux aarch64, gcc 13.3, GNU gdb 15.1, valgrind 3.22.0.

program	bug	question
`null.c`	dereference a NULL pointer	Q1–Q3
`leak.c`	`malloc` without `free`	Q4
`overflow.c`	write `data[100]` in a 100-element array	Q5
`use_after_free.c`	read an element after `free`	Q6
`bad_free.c`	`free` a pointer into the middle of the array	Q7
`vector.c`	(no bug) a `realloc`-based vector vs a linked list	Q8

Q1

> First, write a simple program called `null.c` that creates a pointer to an integer, sets it to `NULL`, and then tries to dereference it. Compile this into an executable called `null`. What happens when you run this program?

```
int *p = NULL;
printf("about to dereference a NULL pointer\n");
fflush(stdout);
printf("value: %d\n", *p);
```

```
$ ./null
about to dereference a NULL pointer
Segmentation fault (core dumped)
$ echo $?
139
```

The program **crashes with a segmentation fault**. Address 0 is not mapped in the process's address space (Linux deliberately leaves the lowest pages unmapped to catch exactly this bug). So the load from `*p` makes the hardware raise a fault, and the kernel kills the process with the signal **SIGSEGV**.

The exit status 139 is 128 + 11, i.e. killed by signal 11 (SIGSEGV). The second `printf` never runs, and the shell gives no hint about *where* in the program it happened.

Q2

> Next, compile this program with symbol information included (with the `-g` flag). Doing so let's put more information into the executable, enabling the debugger to access more useful information about variable names and the like. Run the program under the debugger by typing `gdb ./null` and then, once gdb is running, typing `run`. What does gdb show you?

```
$ gdb ./null
(gdb) run
about to dereference a NULL pointer

Program received signal SIGSEGV, Segmentation fault.
0x0000aaaaaaaa088c in main (argc=1, argv=0xffffffffba8) at null.c:8
8       printf("value: %d\n", *p);
(gdb) bt
#0  0x0000aaaaaaaa088c in main (argc=1, argv=0xffffffffba8) at null.c:8
(gdb) print p
$1 = (int *) 0x0
```

gdb catches the signal **before the process dies** and shows exactly where it happened: the **file and line** (`null.c:8`), the **source line itself**, the function (`main`) and its arguments. Because the program was built with `-g`, I can also inspect variables: `print p` shows that `p` is `0x0`, so the cause is obvious. `bt` (backtrace) shows the chain of function calls; here it is just `main`. Without `-g`, gdb could only show the raw address `0x…088c`.

Q3

> Finally, use the `valgrind` tool on this program. We'll use `memcheck` that is a part of valgrind to analyze what happens. Run this by typing in the following: `valgrind --leak-check=yes ./null`. What happens when you run this? Can you interpret the output from the tool?

```
$ valgrind --leak-check=yes ./null
==4928== Memcheck, a memory error detector
about to dereference a NULL pointer
==4928== Invalid read of size 4
==4928==    at 0x10888C: main (null.c:8)
==4928==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==4928==
==4928== Process terminating with default action of signal 11 (SIGSEGV)
==4928==  Access not within mapped region at address 0x0
==4928==    at 0x10888C: main (null.c:8)
...
==4928== HEAP SUMMARY:
==4928==     in use at exit: 4,096 bytes in 1 blocks
==4928==   total heap usage: 1 allocs, 0 frees, 4,096 bytes allocated
==4928== LEAK SUMMARY:
==4928==    definitely lost: 0 bytes in 0 blocks
==4928==    still reachable: 4,096 bytes in 1 blocks
==4928== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

The program still crashes, but valgrind explains it. `==4928==` is the process ID. Reading the output top to bottom:

- **`Invalid read of size 4 … at main (null.c:8)`**: the program read 4 bytes (one `int`) at line 8.
- **`Address 0x0 is not stack'd, malloc'd or (recently) free'd`**: the address it read, 0, does not belong to the stack, the heap, or any recently freed block, i.e. it is not memory the program owns.
- **`Process terminating with … signal 11 (SIGSEGV)`**: the read crashed the program, as in Q1.
- **HEAP SUMMARY / LEAK SUMMARY**: at the moment it died, 4,096 bytes were still allocated. That is the buffer `printf` allocated for stdout, not a bug in my code. It is "still reachable" (a pointer to it still existed), which is harmless, and nothing was "definitely lost".
- **`ERROR SUMMARY: 1 errors`**: one memory error, the NULL read.

Q4

> Write a simple program that allocates memory using `malloc()` but forgets to free it before exiting. What happens when this program runs? Can you use `gdb` to find any problems with it? How about `valgrind` (again with the `--leak-check=yes` flag)?

```c
int *data = malloc(10 * sizeof(int));
for (int i = 0; i < 10; i++)
    data[i] = i;
printf("data[9] = %d; exiting without free()\n", data[9]);
return 0;                                   // no free(data)
```

- **Running it:** nothing visible happens. It prints `data[9] = 9`, exits with status 0, and the OS reclaims all of its memory when the process ends.
- **gdb:** finds nothing. The program runs to completion (`[Inferior 1 (process …) exited normally]`, and `bt` says `No stack.`). A leak is not a crash; there is nothing for a debugger to stop on.
- **valgrind:** finds it immediately:

```
==5015== HEAP SUMMARY:
==5015==     in use at exit: 40 bytes in 1 blocks
==5015==   total heap usage: 2 allocs, 1 frees, 4,136 bytes allocated
==5015== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==5015==    at 0x4885250: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-arm64-lin
ux.so)
==5015==    by 0x1087EF: main (leak.c:6)
==5015== LEAK SUMMARY:
==5015==    definitely lost: 40 bytes in 1 blocks
==5015== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

The **40 bytes** (10 `int`s) allocated by `malloc` at **`leak.c:6`** were "**definitely lost**": no pointer to them existed any more at exit. The other allocation (4,096 bytes for `printf`'s buffer) was freed this time, so only the real leak is reported. In a short program a leak is harmless, but in a long-running one (a server, a browser) leaks pile up until the process runs out of memory.

Q5

> Write a program that creates an array of integers called `data` of size 100 using `malloc`; then, set `data[100]` to zero. What happens when you run this program? What happens when you run this program using `valgrind`? Is the program correct?

```c
int *data = malloc(100 * sizeof(int));
data[100] = 0;                  // valid indices are 0..99
printf("wrote data[100] = %d\n", data[100]);
free(data);
```

- **Running it:** it **seems to work**. It prints `wrote data[100] = 0` and exits with status 0, every time I ran it. Neither the compiler nor `gdb` notices anything.
- **valgrind** reports two errors:

```
==5030== Invalid write of size 4
==5030==    at 0x10883C: main (overflow.c:7)
==5030==  Address 0x4a7e1d0 is 0 bytes after a block of size 400 alloc'd
==5030==    at 0x4885250: malloc (...)
==5030==    by 0x10882F: main (overflow.c:6)
==5030== Invalid read of size 4
==5030==    at 0x108848: main (overflow.c:8)
==5030==  Address 0x4a7e1d0 is 0 bytes after a block of size 400 alloc'd
==5030== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
```

The write at line 7 and the read at line 8 touch an address "**0 bytes after a block of size 400**", i.e. just past the end of the 400-byte array (100 × 4) that was allocated at line 6.

The program is not correct, even though it runs. `data[100]` is outside the array. The write lands in whatever memory happens to follow the block: here, probably unused padding or the next chunk's header in malloc's own data structures. In a bigger program this silently corrupts other data or the heap itself, and causes a crash much later, somewhere unrelated. Such bugs are very hard to find without a tool like valgrind.

Q6

> Create a program that allocates an array of integers (as above), frees them, and then tries to print the value of one of the elements of the array. Does the program run? What happens when you use `valgrind` on it?

```c
int *data = malloc(100 * sizeof(int));
for (int i = 0; i < 100; i++)
    data[i] = i;
free(data);
printf("data[50] after free = %d\n", data[50]);
```

- **Running it:** yes, it runs and even prints the "right" value, `data[50] after free = 50`, with exit status 0. `free` gave the block back to the allocator, but the bytes were not erased and the memory is still mapped, so reading it does not crash.
- **gcc** actually warns at compile time in this simple case: `warning: pointer 'data' used after 'free' [-Wuse-after-free]`.
- **valgrind:**

```
==5045== Invalid read of size 4
==5045==    at 0x10887C: main (use_after_free.c:10)
==5045==  Address 0x4a7e108 is 200 bytes inside a block of size 400 free'd
==5045==    at 0x48882A8: free (...)
==5045==    by 0x108873: main (use_after_free.c:9)
==5045==  Block was alloc'd at
==5045==    at 0x4885250: malloc (...)
==5045==    by 0x10882F: main (use_after_free.c:6)
==5045== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

valgrind gives the whole story: the read at **line 10** is "200 bytes inside a block of size 400" (`data[50]`, 50 × 4 bytes), the block was **freed at line 9** and originally **allocated at line 6**. Getting the expected value was pure luck; as soon as another `malloc` reuses that memory, `data[50]` would return someone else's data, and writing through it would corrupt them.

Q7

> Now pass a funny value to `free` (e.g., a pointer in the middle of the array you allocated above). What happens? Do you need tools to find this type of problem?

```c
int *data = malloc(100 * sizeof(int));
free(data + 50);
```

```
$ ./bad_free
calling free(data + 50)
free(): invalid pointer
Aborted (core dumped)
$ echo $?
134
```

The program **aborts**. `free` looks for its bookkeeping header just before the pointer it is given. `data + 50` is not the start of a block, so glibc's sanity checks find garbage there, print `free(): invalid pointer` and call `abort()` (signal SIGABRT, exit status 128 + 6 = 134).

Do you need tools? Less than for Q5 and Q6, because this bug announces itself:

- **gcc warns at compile time:** `warning: 'free' called on pointer 'data' with nonzero offset 200 [-Wfree-nonheap-object]`.
- **glibc catches it at run time** and kills the program immediately.
- **gdb** stops on the abort, and the backtrace leads from `abort` / `malloc_printerr` in the C library down to `#8 … in main (…) at bad_free.c:9`, the bad call.

- **valgrind** reports `Invalid free() … Address 0x4a7e108 is 200 bytes inside a block of size 400 alloc'd … by main (bad_free.c:6)`, and also that the 400-byte block is then **definitely lost**, since it was never freed correctly.

Tools are still very useful: the C library's checks are not guaranteed to catch every bad pointer (some just corrupt the heap and crash later), and only gdb or valgrind tell you *which line* passed the bad pointer and where the block came from.

Q8

> Try out some of the other interfaces to memory allocation. For example, create a simple vector-like data structure and related routines that use `realloc()` to manage the vector. Use an array to store the vectors elements; when a user adds an entry to the vector, use `realloc()` to allocate more space for it. How well does such a vector perform? How does it compare to a linked list? Use `valgrind` to help you find bugs.

`vector.c` implements a vector of `int`s with `vector_init`, `vector_push` and `vector_free`. When the array is full, `vector_push` **doubles the capacity** with `realloc`:

```
if (v->size == v->capacity) {
    size_t cap = v->capacity ? 2 * v->capacity : 4;
    int *p = realloc(v->data, cap * sizeof(int));
    if (p == NULL) { perror("realloc"); exit(1); }
    v->data = p;
    v->capacity = cap;
}
v->data[v->size++] = x;
```

Two details matter: the result of `realloc` goes into a temporary pointer first (if `realloc` fails and returns NULL, writing it straight into `v->data` would lose the old array, a leak), and the capacity **doubles** instead of growing by 1. For comparison, the same program builds a singly linked list with one `malloc` per element (keeping a tail pointer so that appends are O(1)), then both are summed and freed.

Results (built with `-O2`):

n		append	`realloc` / `malloc` calls	traverse (sum)	memory for the elements
1,000,000	vector	**1.0 ms**	19 reallocs	**0.3 ms**	4.2 MB (capacity 1,048,576)
	linked list	14.5 ms	1,000,000 mallocs	1.7 ms	16 MB of nodes
10,000,000	vector	**7.7 ms**	23 reallocs	**3.1 ms**	67 MB (capacity 16,777,216)
	linked list	146.6 ms	10,000,000 mallocs	12.3 ms	160 MB of nodes

The vector performs very well: 15–19× faster to build and 4–6× faster to traverse than the linked list:

- **Doubling makes appends amortized O(1).** 10 million appends need only 23 `realloc` calls. A `realloc` may have to copy the whole array, but the copies add up to fewer than 2n element copies in total. Growing by one element at a time would instead copy the array on almost every append ($O(n^2)$ overall).
- **The linked list calls `malloc` once per element**, and each call costs far more than writing one `int` into an array.
- **Traversal:** the vector's elements are contiguous, so the CPU's caches and prefetcher work perfectly. The list has to follow a pointer to a separate heap block for every element.
- **Memory:** an `int` takes 4 bytes in the vector (up to 2× slack from doubling), but each list node needs 16 bytes (4 for the value, 4 of padding, 8 for the `next` pointer). In reality each node uses even more, because malloc adds its own header to every block.

The linked list wins only for operations this benchmark does not do: inserting or removing in the middle (O(1) with a pointer to the node, versus shifting elements in the vector), and keeping pointers to elements stable (a `realloc` can move the whole array, invalidating any pointer into it).

valgrind (run with n = 100,000, since valgrind slows programs down a lot):

```
==5094== HEAP SUMMARY:
==5094==     in use at exit: 0 bytes in 0 blocks
==5094==   total heap usage: 100,017 allocs, 100,017 frees, 2,652,656 bytes allocated
==5094== All heap blocks were freed -- no leaks are possible
==5094== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

No errors and no leaks. Every allocation was freed. The 100,017 allocations are the 100,000 list nodes, 16 `realloc`s of the vector, and `printf`'s buffer.

Q9

> Spend more time and read about using `gdb` and `valgrind`. Knowing your tools is critical; spend the time and learn how to become an expert debugger in the UNIX and C environment.

The commands I found most useful while doing this homework:

tool	command	what it does
gdb	`run` / `r`	start the program (add arguments after it)
	`bt`	backtrace: the chain of function calls at the crash
	`break file.c:N` / `b func`	stop at a line or function
	`next` / `step` / `continue`	run one line (over / into calls), or until the next breakpoint
	`print expr` / `p *ptr@10`	print a variable, or 10 elements starting at `ptr`
	`frame N`, `info locals`	move to a caller in the backtrace and see its variables
	`watch var`	stop whenever a variable changes
valgrind	`valgrind --leak-check=full ./prog`	memory errors plus a detailed leak report
	`--show-leak-kinds=all`	also list "still reachable" blocks
	`--track-origins=yes`	show where an uninitialized value came from
gcc	`-g`	debug symbols, so both tools can show file names and line numbers
	`-Wall`	warnings; caught the use-after-free and the bad `free` at compile time
	`-fsanitize=address`	AddressSanitizer: detects overflows and use-after-free at run time, much faster than valgrind

Summary of what each tool caught in this homework:

bug	runs "fine"?	gcc `-Wall`	gdb	valgrind
NULL dereference (Q1–3)	no, segfault	–	✅ exact line	✅ invalid read of address 0
memory leak (Q4)	yes	–	❌	✅ 40 bytes definitely lost
overflow `data[100]` (Q5)	**yes**	–	❌	✅ invalid write/read past the block
use after free (Q6)	**yes**	✅ warning	❌	✅ invalid read inside a freed block
`free` of a middle pointer (Q7)	no, abort	✅ warning	✅ via backtrace	✅ invalid free + leak

The most dangerous bugs are the ones that **run fine** (Q5, Q6): only valgrind found every one of them.